Word Embeddings

Clase 3 · Procesamiento de Lenguaje Natural

Andrés Abeliuk, Fabrizzio Pezzolla

Diplomado en Inteligencia Artificial · DCC · Universidad de Chile

Dónde quedamos: vectores largos y dispersos

Clase 2

Una palabra = una columna de la matriz documento-término (TF-IDF).

Comparamos palabras por los documentos que comparten.

El problema

Decenas de miles de dimensiones, casi todas cero.

“cuerda” y “cordel” casi no se conectan si nunca comparten documento.

Hoy

Vectores cortos (50–1000) y densos que se aprenden prediciendo palabras.

Funcionan mejor que los dispersos en casi todas las tareas.

De contar contextos a predecirlos

Llevé a mi ____ al veterinario

¿Qué palabras pueden ir en el hueco?

perro | **gato** | **conejo** | **loro**

Si un modelo aprende a predecir el hueco, aprende qué palabras se usan de forma parecida

Qué vamos a poder explicar

01 Por qué un vector denso aprendido es mejor que uno disperso

02 Cómo word2vec (skip-gram y CBOW) aprende vectores desde texto crudo

03 Qué propiedades tienen: similitud, analogías… y sesgos

04 Cuándo usarlos y cuáles son sus límites (fastText, polisemia)

De dispersos a densos

Aprender la representación en vez de diseñarla

Aprendizaje de representaciones

- **Antes:** nosotros elegíamos las características (conteos, TF-IDF).
- **Ahora:** el modelo descubre automáticamente las características necesarias para resolver una tarea.
- Cada palabra se ubica en un espacio donde la cercanía refleja parecido de uso, semántico y sintáctico.



Figura: Jurafsky & Martin, Speech and Language Processing, cap. Embeddings.

Embeddings: vectores cortos y densos

50–1000

dimensiones (no |V|)

$\mathbb{R}$

valores reales, casi ninguno es cero

?

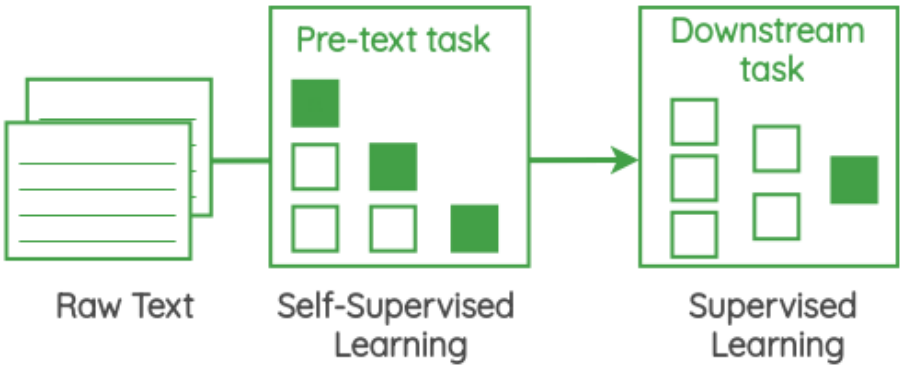
ninguna dimensión tiene un significado propio

- **Ventajas:** rápidos de entrenar; hay modelos preentrenados disponibles; sirven como entrada para otros modelos.
- **Limitación:** un vector fijo por palabra, sin importar el contexto (lo retomamos al final).

word2vec

Skip-gram, CBOW y muestreo negativo (Mikolov et al., 2013)

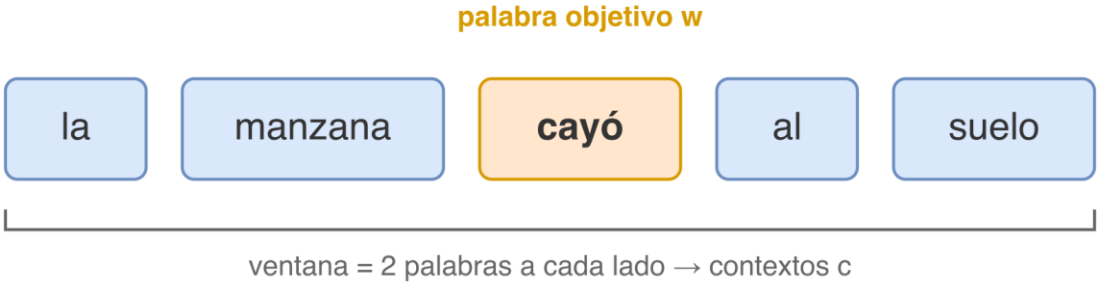
Autosupervisión: el texto es su propia etiqueta



- El texto crudo define una tarea supervisada implícita: ¿qué palabras aparecen cerca de cuál?
- Las respuestas correctas están en el propio corpus: no hay que etiquetar nada.
- Lo aprendido en esa tarea pretexto se reutiliza en tareas con pocas etiquetas.

Esquema: tarea pretexto autosupervisada → tarea final supervisada.

La ventana de contexto fabrica los ejemplos



Ejemplos positivos (+)

- (cayó, la)
- (cayó, manzana)
- (cayó, al)
- (cayó, suelo)

Ejemplos negativos (–): c al azar

- (cayó, impuesto)
- (cayó, guitarra)
- (cayó, azul)
- (cayó, lunes)

En el notebook: pares_skipgram() y ejemplos_cbow() sobre esta misma oración.

Skip-gram: un clasificador binario

- Dado un par (w, c), ¿es c un contexto real de w?
  - (manzana, cayó) → 1 (manzana, perro) → 0
- Similitud ≈ producto punto c · w
- La sigmoide convierte el producto punto en probabilidad

$$P(+|w,c) = \sigma(\mathbf{c}\cdot\mathbf{w}) = \frac{1}{1+\exp(-\mathbf{c}\cdot\mathbf{w})}$$

No nos interesa el clasificador: nos quedamos con sus pesos como embeddings

Ecuación: Jurafsky & Martin, cap. Embeddings.

Varias palabras de contexto

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(\mathbf{c_i} \cdot \mathbf{w})$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(\mathbf{c_i} \cdot \mathbf{w})$$

- Suposición simplificadora: las palabras de contexto son independientes.
- Multiplicamos sus probabilidades; en la práctica sumamos logaritmos.
- El orden dentro de la ventana no importa.

Muestreo negativo

Cada par positivo se acompaña de k pares negativos



¿Cómo se eligen los negativos?

Según su frecuencia ponderada:

$$P(w) \propto \text{frecuencia}(w)^{0,75}$$

El exponente 0,75 da algo más de probabilidad a palabras poco frecuentes que la frecuencia cruda.

gensim: negative=k (típico 5 a 20)

Función objetivo

Skip-gram original: softmax sobre todo el vocabulario

El denominador recorre las |V| palabras (decenas de miles) en cada ejemplo → muy caro

$$P(c \mid w) = \frac{e^{\mathbf{c} \cdot \mathbf{w}}}{\sum_{v \in V} e^{\mathbf{v} \cdot \mathbf{w}}}$$

Muestreo negativo: un par real y k negativos

$$L = -\left[\log \sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{neg_i} \cdot \mathbf{w}) \right]$$

Minimizar L = acercar el contexto real (σ → 1) y alejar k palabras al azar (σ → 0): solo k + 1 palabras por ejemplo

Mikolov et al. (2013a) Efficient Estimation…; (2013b) Distributed Representations of Words and Phrases…

Gradiente: predicción – etiqueta

Para cada ejemplo (w, c) con etiqueta y = 1 (real) o y = 0 (negativo):

$$L = -[\, y \log \sigma(\mathbf{c} \cdot \mathbf{w}) + (1 - y) \log (1 - \sigma(\mathbf{c} \cdot \mathbf{w})) \,]$$

$$\frac{\partial L}{\partial (\mathbf{c} \cdot \mathbf{w})} = \sigma(\mathbf{c} \cdot \mathbf{w}) - y \;=\; \text{error}$$

$$\frac{\partial L}{\partial \mathbf{w}} = \text{error} \cdot \mathbf{c} \qquad \frac{\partial L}{\partial \mathbf{c}} = \text{error} \cdot \mathbf{w}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \text{error} \cdot \mathbf{c} \qquad \mathbf{c} \leftarrow \mathbf{c} - \eta \, \text{error} \cdot \mathbf{w}$$

y = 1, σ = 0,3

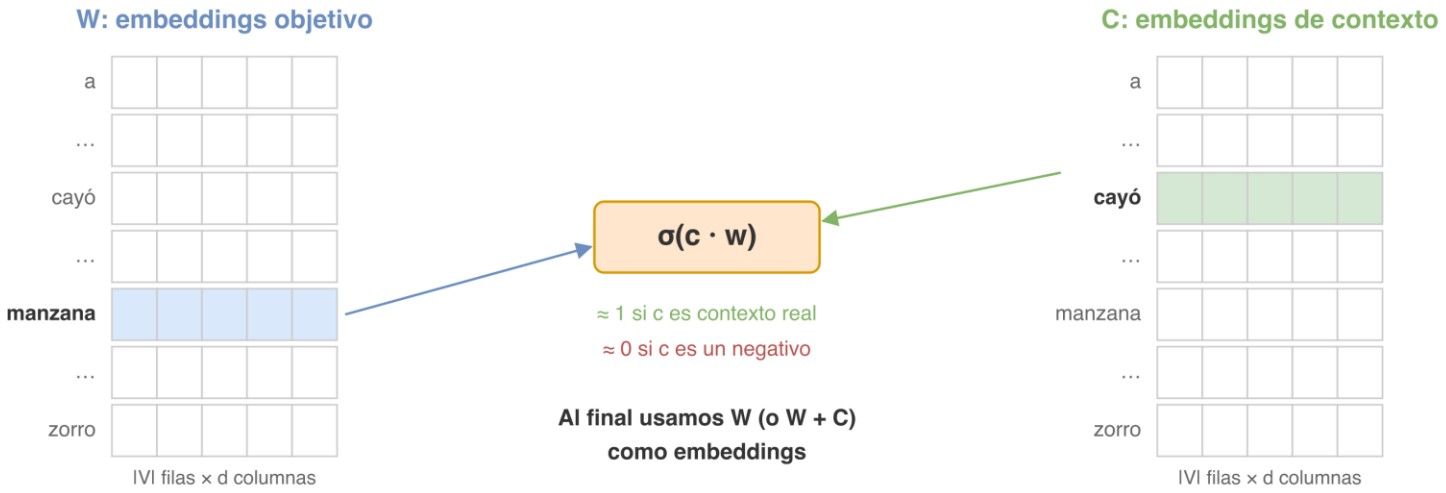
error = −0,7
→ w se acerca a c

y = 0, σ = 0,8

error = +0,8
→ w se aleja de c

Simplificado: actualizamos por ejemplo con η fijo; word2vec acumula el gradiente del par real y los k negativos, con η decreciente.

Aprendizaje: acercar y alejar



- Vectores iniciales al azar; descenso de gradiente sobre muchos pares.
- Acercar w a contextos reales (c·w ↑) y alejarlo de los negativos (c·w ↓).

Un paso de gradiente

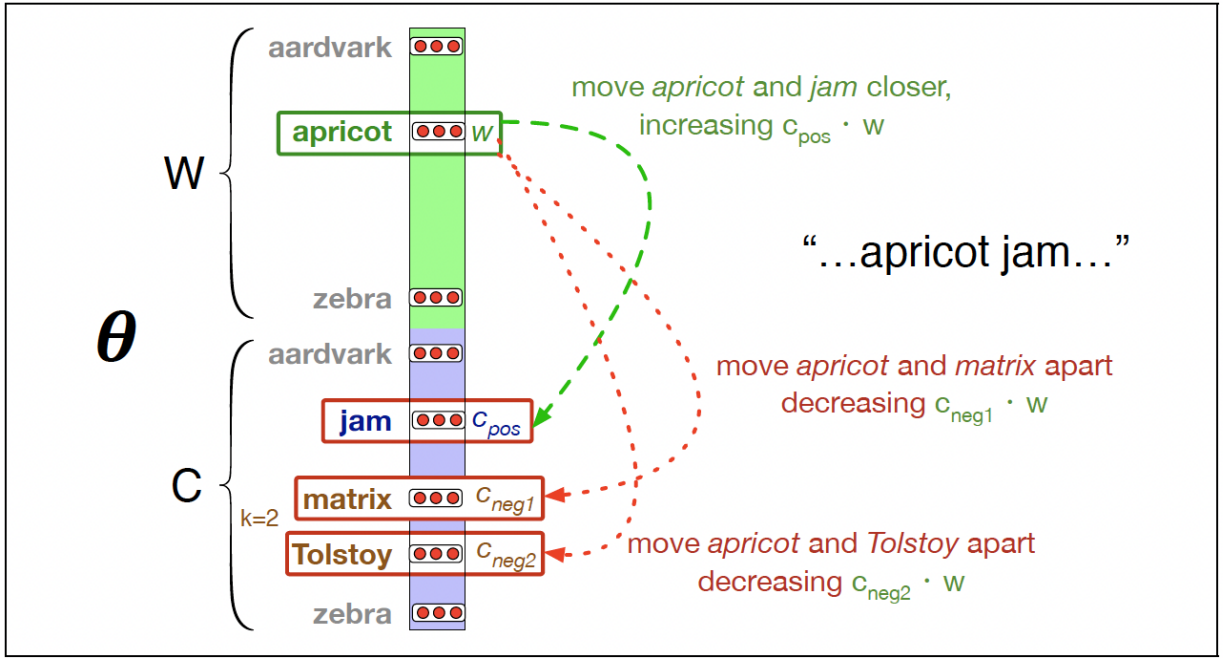
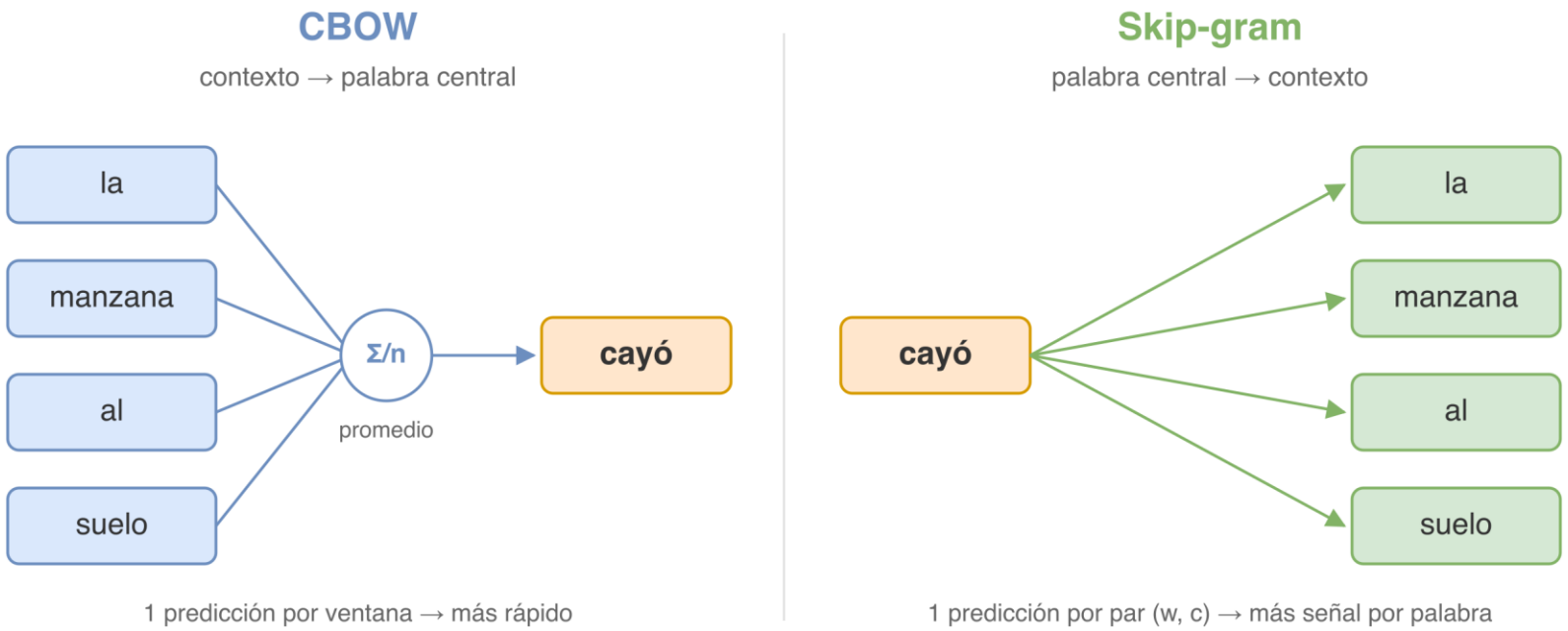


Figura: Jurafsky & Martin, Speech and Language Processing, cap. Embeddings.

CBOW: la otra dirección



Skip-gram vs. CBOW

Aspecto	CBOW	Skip-gram
Tarea	Contexto → palabra central	Palabra central → contexto
Ejemplos por ventana	1 (contexto promediado)	1 por cada vecino
Velocidad	Más rápido	Más lento
Palabras frecuentes	Muy buenas	Buenas
Palabras poco frecuentes	Peores	Mejores
En gensim	sg=0	sg=1

Lo comprobamos con 200.000 reseñas en el notebook

Límites y extensiones

Polisemia, palabras nuevas y modelos preentrenados

Embeddings estáticos y polisemia

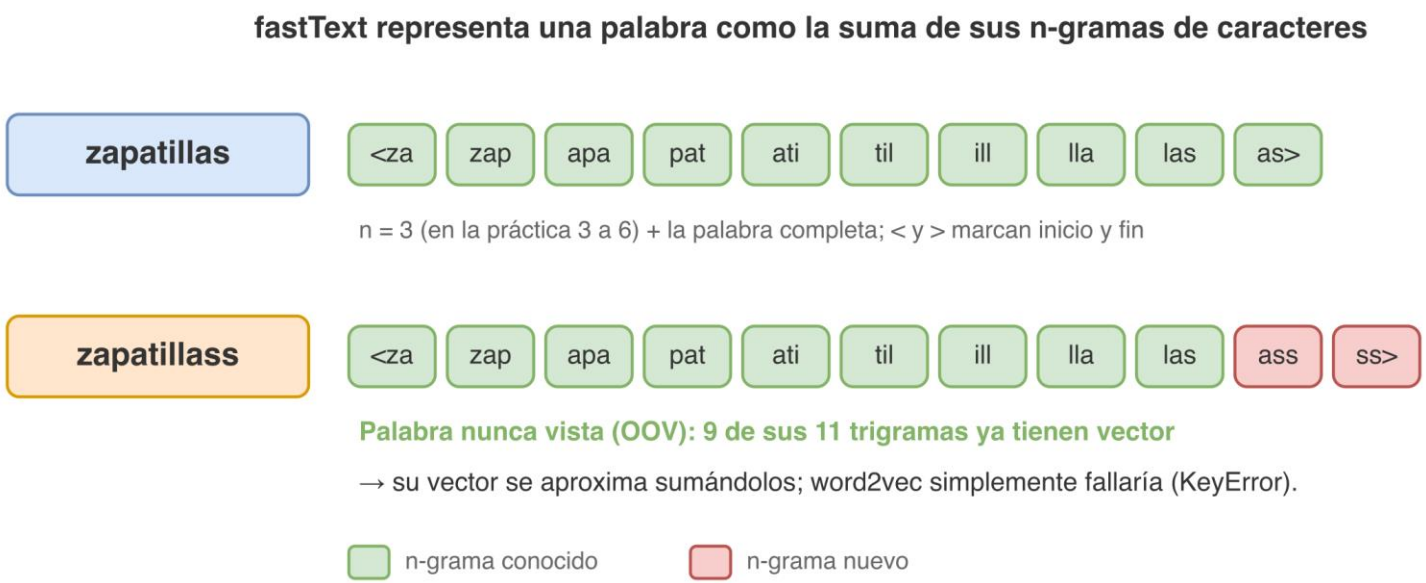
Me senté en el banco de la plaza

Pedí un préstamo al banco

un único vector para “banco”

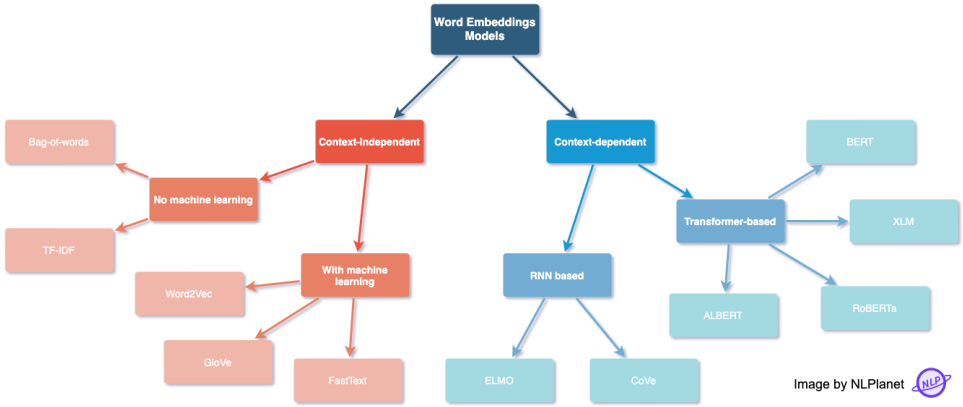
- El vector mezcla todos los sentidos, ponderados por su frecuencia en el corpus.
- Los embeddings contextuales (transformers) asignan un vector distinto a cada aparición.

fastText: subpalabras



Representaciones preentrenadas

- **word2vec, GloVe, fastText:** vectores públicos entrenados con miles de millones de palabras.
- **Español:** SBWCE (Cardellino, 2016), fastText (Grave et al., 2018).
- Se usan como entrada o como inicialización de redes neuronales.
- Elegir el corpus importa: dominio general vs. dominio de la tarea.



Ahora en Python

1–2 Matriz término-término · fabricar pares skip-gram y CBOW

3 Skip-gram en miniatura: gradiente y vectores 2D

4–5 gensim · word2vec en 200.000 reseñas: skip-gram vs. CBOW

6–7 Similitud · PCA, t-SNE y el eje de género

8–9 Analogías (evaluación intrínseca) y sesgos

10–11 fastText y OOV · clasificar reseñas (extrínseca)

Resumen

- Un embedding es un vector corto y denso; sus direcciones capturan parecidos de uso.
- word2vec aprende sin etiquetas: predice contextos (skip-gram) o la palabra central (CBOW).
- Clasificador $\sigma(c \cdot w)$ con muestreo negativo: acerca contextos reales, aleja negativos.
- Propiedades: vecinos, analogías (paralelogramo)… y sesgos del corpus.
- fastText resuelve palabras nuevas; la polisemia queda para los embeddings contextuales.

Lecturas

- **Jurafsky & Martin.** Speech and Language Processing, 3.ª ed., cap. Embeddings.
- **Mikolov et al. (2013).** Efficient Estimation of Word Representations in Vector Space.
- **Bojanowski et al. (2017).** Enriching Word Vectors with Subword Information.
- **Bolukbasi et al. (2016).** Man is to Computer Programmer as Woman is to Homemaker?
- **Cardellino (2016).** Spanish Billion Words Corpus and Embeddings.